

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	N.KALYAN REDDY	Reg. No.:	192424232
Date:	13-08-2026	Duration:	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a Code Review and Repository Evaluation of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

1. Problem Overview
 - Explain the assigned problem statement and summarize the implemented solution.
 - Describe the project objectives and key functionalities.
2. Repository Organization
 - Evaluate the repository structure, folder organization, and file naming conventions.
 - Justify how the repository layout supports maintainability and collaboration.
3. Code Quality Review
 - Review the source code for readability, modularity, coding standards, and documentation.
 - Identify strengths and areas for improvement.
4. Version Control Evaluation
 - Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
 - Explain how version control supports collaborative software development.
5. Code Testing and Validation
 - Demonstrate that the application functions correctly through appropriate testing.
 - Present test cases, execution results, and screenshots as evidence.
6. Repository Documentation

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
 - Suggest improvements to enhance usability and maintainability.
7. Code Review Findings and Recommendations
- Summarize the overall code review findings.
 - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15%)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository	README, installation guide, usage instructions, and documentation are	Documentation is mostly complete.	Basic documentation	Poor or insufficient documentation.

Maintenance(15%)	complete, clear, and well organized.		with missing details.	
Review Findings, Recommendations & Presentation(10%)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

1. Problem Overview

Problem Statement

The Intelligent AI-Based Smart Parking and Vehicle Guidance System is designed to solve parking shortages, traffic congestion, and inefficient parking management in urban areas. The system uses AI, IoT sensors, cameras, GPS, and cloud computing to provide real-time parking availability and guide drivers to vacant parking spaces.

The system detects vacant parking spaces, predicts parking demand, provides parking reservations, supports digital payments, monitors occupancy, and generates parking utilization reports.

Implemented Solution

The proposed solution consists of a frontend application, backend server, AI modules, database, IoT sensors, and cameras. Parking availability is detected using cameras and IoT sensors. The backend processes the information and provides it to users through the frontend. AI is used for parking demand prediction, while the database stores parking, user, reservation, and occupancy information.

Project Objectives

1. Detect parking availability automatically.
2. Guide drivers to vacant spaces.
3. Predict parking demand using AI.
4. Automate parking reservations.
5. Support digital payment integration.
6. Monitor parking occupancy.
7. Generate parking utilization reports.
8. Reduce urban traffic congestion.

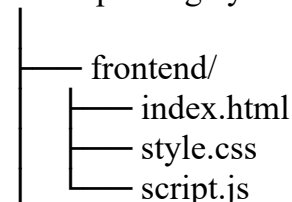
Key Functionalities

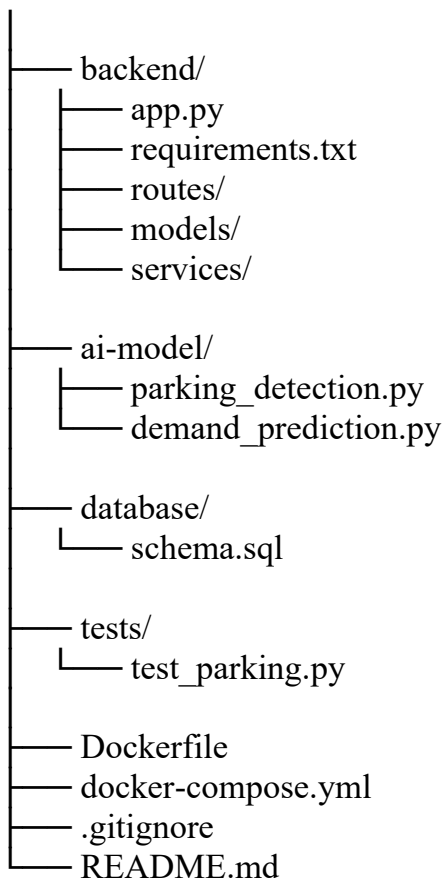
- Real-time parking space detection.
- Parking availability display.
- Vehicle guidance to vacant spaces.
- AI-based parking demand prediction.
- Parking reservation.
- Digital payment support.
- Parking occupancy monitoring.
- Parking utilization reporting.

2. Repository Organization

Repository Structure

smart-parking-system/





Folder Organization

- frontend/ – Contains the user interface.
- backend/ – Contains server-side code and APIs.
- ai-model/ – Contains AI-based parking detection and demand prediction modules.
- database/ – Contains database-related files.
- tests/ – Contains testing files.
- Dockerfile – Defines the containerization instructions.
- docker-compose.yml – Defines the application services.
- .gitignore – Specifies files that should not be tracked by Git.
- README.md – Contains project documentation.

File Naming Conventions

Files use clear and meaningful names such as:

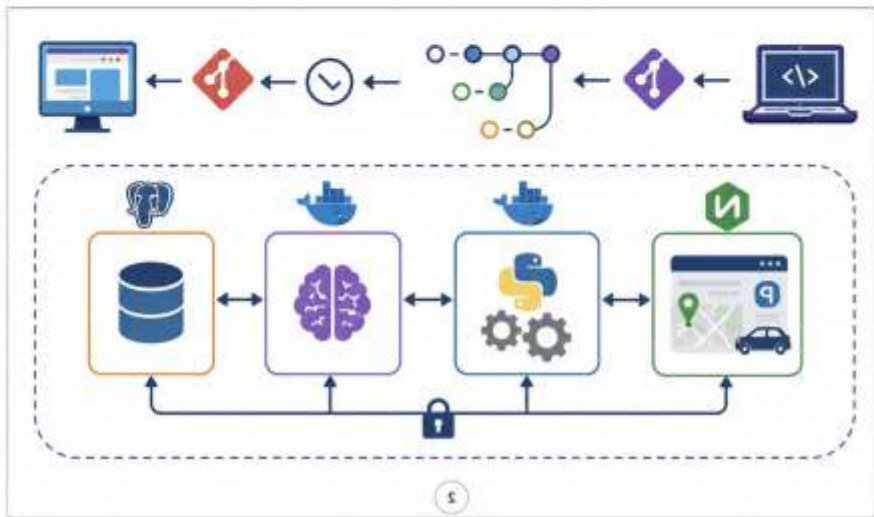
parking_detection.py

demand_prediction.py

test_parking.py

docker-compose.yml

The repository layout supports maintainability because related files are grouped into separate folders. It also supports collaboration because developers can work independently on frontend, backend, AI, database, and testing components.



3. Code Quality Review

Readability

The source code should use meaningful variable names, clear function names, proper indentation, and appropriate comments.

Example:

```
def get_available_spaces(parking_data):
    available_spaces = []

    for space in parking_data:
        if space["status"] == "available":
            available_spaces.append(space)
```

```
    return available_spaces
```

The function name `get_available_spaces()` clearly describes its purpose.

Modularity

The project is divided into separate modules:

- Frontend module.
- Backend module.
- AI module.
- Database module.
- Testing module.

This makes the application easier to understand, test, modify, and maintain.

Coding Standards

The project follows basic coding practices such as:

- Meaningful variable names.
- Meaningful function names.
- Proper indentation.
- Modular programming.
- Comments for important logic.
- Separate files for different functionalities.

Documentation

Important functions and modules should contain comments or documentation explaining their purpose.

Strengths

1. Clear project structure.
2. Modular source code.
3. Meaningful file names.
4. Separation of frontend and backend.
5. Separate AI modules.
6. Dedicated testing folder.
7. Docker configuration included.

Areas for Improvement

1. Add more detailed documentation.

2. Add stronger input validation.
3. Improve error handling.
4. Increase automated test coverage.
5. Add documentation for important functions.
6. Follow consistent coding standards throughout the project.

4. Version Control Evaluation

Commit History

A suitable commit history for the project is:

Initial project setup

Add frontend parking interface

Add backend API

Add parking detection module

Add AI demand prediction

Add reservation functionality

Add digital payment functionality

Add vehicle guidance feature

Add Docker configuration

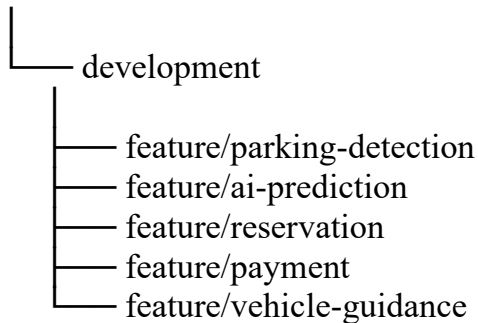
Fix parking availability calculation

Update project documentation

These commit messages clearly describe the changes made to the project.

Branching Strategy

main



The main branch contains the stable version, while development is used for integrating new changes. Feature branches are used to develop individual functionalities.

Repository Maintenance

Repository maintenance includes:

- Regular commits.
- Meaningful commit messages.
- Proper branch management.
- Removing unnecessary files.
- Maintaining .gitignore.
- Updating the README.
- Synchronizing the local and remote repositories.

5. Code Testing and Validation

Test Cases

Test Case	Input	Expected Result
Parking availability	Parking location	Available spaces displayed
Parking detection	Camera/sensor data	Occupied/vacant status detected
Parking reservation	Selected space	Reservation successfully created
Demand prediction	Historical parking data	Predicted parking demand displayed
Digital payment	Valid payment details	Payment processed successfully
Vehicle guidance	Selected parking space	Route/guidance displayed
Occupancy monitoring	Parking sensor data	Occupancy status updated

Testing Procedure

1. Start the application.
2. Check the parking availability module.
3. Verify vacant and occupied spaces.
4. Test parking reservation.
5. Test AI demand prediction.
6. Test digital payment functionality.
7. Test vehicle guidance.
8. Verify that parking information is updated correctly.

Expected Testing Result

The application should correctly detect parking availability, display vacant spaces, process reservations, provide demand predictions, and update parking occupancy information.

The assignment specifically requires test cases, execution results, and screenshots as evidence.

6. Repository Documentation

README

The README file should contain:

- Project title.
- Project description.
- Objectives.
- Technologies used.
- Project structure.

- Installation instructions.
- Execution instructions.
- Docker instructions.
- Testing instructions.

Installation Instructions

```
git clone <repository-url>
cd smart-parking-system
pip install -r backend/requirements.txt
```

Docker Execution

```
docker compose build
docker compose up -d
```

Usage Guide

1. Open the Smart Parking application.
2. Search for a parking location.
3. View available parking spaces.
4. Select a vacant parking space.
5. Reserve the parking space.
6. Complete the digital payment.
7. Follow the vehicle guidance.

Dependencies

The project dependencies include:

- Python.
- Backend framework.
- Database system.
- AI/ML libraries.
- Frontend technologies.
- Docker.

Documentation Improvements

1. Add detailed installation instructions.
2. Add screenshots of the application.
3. Explain API endpoints.
4. Document database configuration.
5. Add troubleshooting instructions.
6. Add testing instructions.

The assignment requires evaluation of the README, installation instructions, usage guide, dependencies, and suggestions for improving usability and maintainability.

7. Code Review Findings and Recommendations

Overall Findings

The Smart Parking System has a modular repository structure and separates the frontend, backend, AI, database, and testing components. Git provides version control and supports collaborative development. Docker can provide a consistent deployment environment.

Code Quality Findings

- The code is organized into separate modules.
- Meaningful file names improve readability.

- Separate AI and backend modules improve maintainability.
- Additional validation and error handling can improve reliability.

Repository Management Findings

- Feature-based branches support parallel development.
- Meaningful commits make project history easier to understand.
- .gitignore helps prevent unnecessary files from being added.
- Regular repository maintenance improves project organization.

Testing Findings

- Functional test cases should cover parking detection, availability, reservation, payment, demand prediction, and guidance.
- Automated testing should be increased.
- Test results should be documented with screenshots.

Recommendations

1. Improve source-code documentation.
2. Increase automated test coverage.
3. Add stronger error handling.
4. Improve input validation.
5. Maintain meaningful Git commit messages.
6. Use feature branches for new functionality.
7. Keep the README updated.
8. Add detailed installation and usage instructions.
9. Improve AI prediction accuracy.
10. Improve parking detection accuracy.
11. Add more real-time monitoring capabilities.
12. Improve future cloud-based deployment and scalability.

The assignment requires the final review to summarize findings and recommend improvements in code quality, repository management, documentation, testing, and future enhancements.